DeFuzz: Agentic Discovery of Silently-Failing Compiler Defenses via Cross-Mechanism Security Invariants

Anonymous Submission
Paper #XXX — Under Review

Abstract—Compiler-inserted defenses—stack canaries, _FORTIFY_SOURCE, CET/IBT, CFI, BTI, PAC, and shadow stacks—are the last line that must hold once a memory-safety bug is triggered. Their effectiveness depends not only on the correctness of the defense code but also on assumptions about the underlying instruction set architecture (ISA). When such an assumption breaks on a particular backend, the defense can fail silently: the program compiles without warning and runs with correct functionality, yet its security contract is no longer enforced. CVE-2023-4039, in which the stack protector of GCC is defeated on AArch64 in the presence of variable-length arrays, is one such case, and its cross-architecture siblings have persisted upstream for years.

The detection of silent failures is challenging due to two coupled reasons. The search space is a two-dimensional matrix of defense mechanism × ISA, each cell backed by an independent middle-end pass or backend template. Furthermore, failure adjudication is difficult: the binary neither crashes nor disagrees with other compilers; thus, crash-based and differential-oracle fuzzers register nothing. We call this the oracle gap.

We present DeFuzz, an agentic system that closes the oracle gap and searches the matrix directly. First, we systematize the safety invariants that each defense must satisfy—distilled from compiler source comments, ABI specifications, and confirmed historical bugs—into a machine-checkable oracle; every checker returns a four-state verdict and stays sound by grounding each bug claim in a deterministic binary or runtime signal rather than in model output. Second, we introduce a cross-mechanism invariant-generation pipeline that treats a confirmed root cause in one mechanism as a probe, retrieves structurally analogous code in another mechanism, and instantiates a new invariant behind two static grounding gates; from 25 confirmed defense bugs and 24 probes over GCC 16.1 it yields 11 new cross-mechanism invariants, none duplicating an existing seed. Third, we build an explicitly-orchestrated agentic loop: a deterministic pipeline drives build, coverage, and oracle in a fixed order and invokes agents only at fixed positions, where they communicate through a versioned blackboard rather than free-form dialogue. Unlike free-running agent systems, this design yields reproducible traces, per-edge ablation, and bug claims that trace back to deterministic evidence; a checker-bound ISA routing scheme collapses the mechanism × ISA Cartesian product into a single semantic decision. On GCC, LLVM, lld, and compiler-rt, DeFuzz discovered 29 silent-failure defects spanning 17 defense mechanisms and 15 instruction-set targets, each grounded in a deterministic binary or runtime signal.

I. Introduction

The number of publicly disclosed vulnerabilities has grown for a decade; the CVE database now registers more than twenty thousand new entries per year and the rate is still rising. Major vendors have conceded that manual auditing and secure-coding disciplines alone are insufficient to eliminate memory-safety bugs in upper-layer code. As a consequence, the behavior of a program following a bug trigger has become increasingly critical: stack canaries, _FORTIFY_SOURCE, Indirect Branch Tracking (IBT), Control-Flow Integrity (CFI), Pointer Authentication (PAC), and shadow stacks are deployed jointly by the compiler and the runtime as a backstop intended to hold even when an exploit fires.

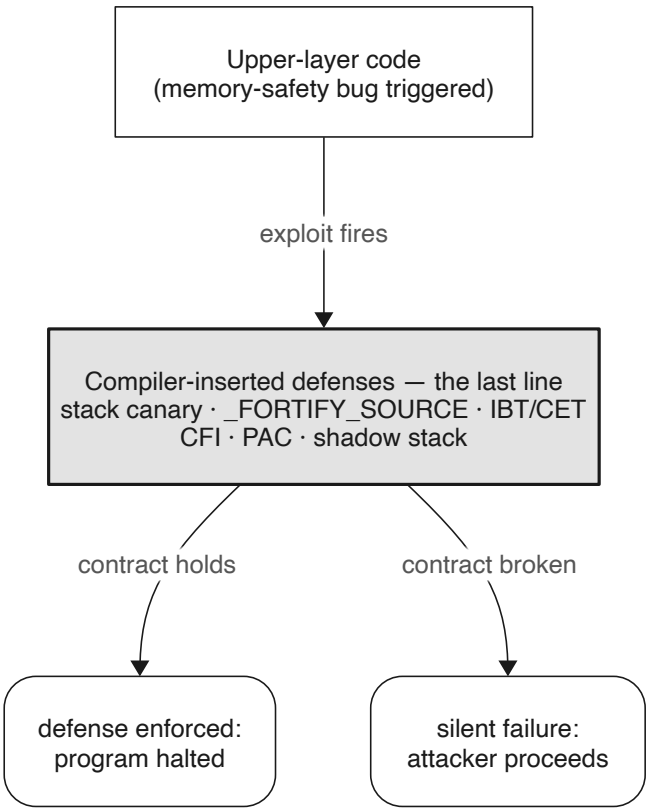

Fig. 1. Compiler-inserted defenses as the runtime last line: they must hold once an upper-layer memory-safety bug is triggered.

Compiler-emitted defenses share a common working model: at compile time the compiler inserts check instructions, layout constraints, or metadata so that any run-time

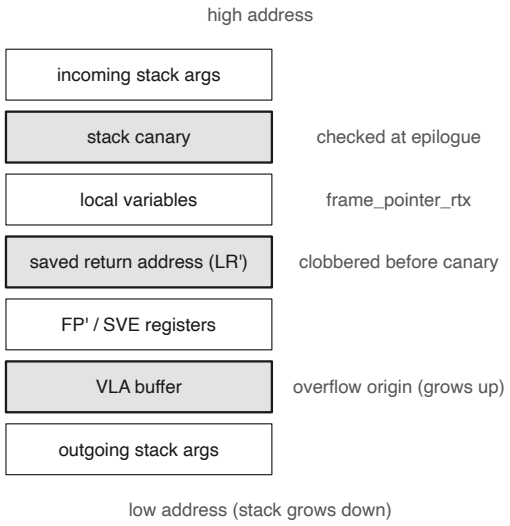

Fig. 2. CVE-2023-4039 buggy stack frame: canary above the VLA, return address below it, so an overflow bypasses the check.

deviation from the intended behavior is caught and the program is terminated. The stack canary is the canonical example—a random sentinel is placed between the buffer and the saved return address, and is compared against its original value before the function returns.

However, defense mechanisms themselves consist of code, which is susceptible to bugs. Furthermore, the effectiveness of a defense depends not only on the correctness of its implementation but also on the underlying ISA. As the number of ISAs keeps growing, the cross-architecture attack surface widens, and the rigor of defense implementations within compilers becomes correspondingly harder to guarantee.

A. Silent failure

CVE-2023-4039 is a representative case. The -fstack-protector mechanism of GCC was defeated on AArch64 for a long time: when a function uses a variable-length array (VLA) or alloca, the compiler places the canary above the VLA while the saved return address sits below it, so an overflow rewrites the return address before the canary is ever checked. The bug had existed since early GCC versions and was only analyzed and fixed by outside researchers in 2023. Figure 2 shows the buggy frame layout.

This class of failure has a double-silent character: it raises no error at compile time (toolchain and functional tests see nothing) and produces correct functional results at run time (program behavior sees nothing), yet the security contract is already broken. We call this a silent failure: the compiled artifact appears to carry a complete line of defense, but an attacker can bypass it entirely.

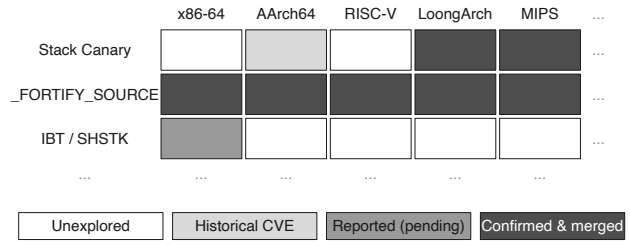

Fig. 3. The defense mechanism × ISA matrix. Each cell is an independent backend/pass; silent failures hide in individual cells.

B. Two coupled challenges

Systematically exposing silent failures presents two intertwined challenges.

Large search space. As shown in Figure 3, the compiler must uphold a security contract across a two-dimensional space of many defense mechanisms (canary, _FORTIFY_SOURCE, IBT, CFI, …) and many ISAs (x86-64, AArch64, RISC-V, LoongArch, …). Each cell maps to an independent middle-end pass or backend template, and any detail can decide whether the mechanism takes effect. Homologous omissions are not rare: PR-96191, a sibling of CVE-2023-4039 fixed in 2020, covered only some architectures and left several fallback backends untouched, so the same failure persisted silently on other targets for years. Our preliminary experiments already confirmed several such homologous failures with the GCC upstream.

Difficulty in failure adjudication: the oracle gap. Even when a test reaches the relevant code path, "is the defense contract still satisfied" is not directly observable: the program does not crash, its output agrees with other compilers, and everything looks normal while the contract has already been violated. We call the static or dynamic properties a defense must satisfy at run time its safety invariants (e.g., the canary must lie between any overflow-reachable stack object and the return address); breaking them is a silent failure. Existing automated methods are oblivious to this phenomenon: crash- or differential-output tools such as Csmith and YARPGen trigger no anomaly, and coverage-guided fuzzing lacks the oracle to judge the security status of each cell.

C. Our approach

DeFuzz closes the oracle gap and searches the matrix directly by combining an invariant-grounded oracle with an explicitly-orchestrated agentic loop. We distill the safety invariants of each defense into machine-checkable checkers that form a sound oracle; we generate new cross-mechanism invariants by retrieval-augmented analogy transfer; and we drive seed generation with an agentic loop whose orchestration is deterministic and whose agents

are grounded by that oracle. A checker-bound ISA routing scheme keeps the agent away from the raw ISA axis.

D. Contributions

- An oracle for silent failure (main line). A sound, machine-checkable oracle that fills the silent-failure oracle gap and supplies false-positive-free ground truth for agentic bug hunting (§VI).
- Systematized security invariants (support). A bottom-up taxonomy over 468 catalogued invariants across more than two dozen defenses, each in a machine-decidable static/dynamic form, plus a cross-mechanism generation pipeline that yields 11 new invariants (§IV, §V).
- An explicitly-orchestrated agent system (moat). A deterministic pipeline with agents at fixed positions linked through a versioned blackboard, yielding reproducible, ablatable, and auditable runs—in contrast to free-running agent systems (§VII).
- An agentic loop (vehicle). Oracle-grounded, coverage-guided seed generation with checker-bound ISA routing that collapses the Cartesian product (§VII).

II. Background and Motivation

A. Compiler defenses and the mechanism × ISA matrix

A compiler-emitted defense inserts, at compile time, either extra check instructions (canary comparison, CFI target checks), layout constraints (guard placement, shadow-stack slots), or metadata (BTI landing pads, PAC signing) so that a run-time deviation is detected and the program is halted. Whether the inserted defense is actually effective is decided on a per-backend basis: stack layout, register allocation, and instruction encoding all differ across ISAs, and each is realized by a distinct pass or target template. The matrix in Figure 3 therefore has genuinely independent cells, and a defense that is correct in one cell can be silently broken in the next.

B. Silent failure: a real case

Returning to CVE-2023-4039: the effectiveness of the stack protector depends on the invariant "the canary lies between every overflow-reachable stack object and the saved return address." On AArch64 with a VLA, that invariant is violated by construction, yet nothing in the build or in a functional test reveals it. PR-96191 demonstrates a recurrence of this pattern: a fix that lands on some targets but not on fallback backends leaves the invariant violated elsewhere. Both cases exemplify the same phenomenon—a complete-looking defense that an attacker bypasses.

C. Why existing methods miss it

Static analysis is constrained by patterns: silent failures exhibit heterogeneous forms, and most confirmed cases involve idiosyncratic violations, rendering pattern matching difficult to generalize. Crash and differential fuzzing monitor surface-level signals—whether the program crashes or disagrees with another compiler—which by construction cannot catch a silent logic failure; this is the oracle gap. Free-running agent fuzzers can find bugs but sacrifice reproducibility: with an agent deciding the control flow, when to compile, and when to declare a bug, trajectories cannot be replayed, ablations cannot be run, and a bug claim cannot be traced back to a deterministic cause.

D. Scope and threat model

We target the compiler's own defense implementation on GCC and LLVM. A finding is a silent failure: a compiled artifact whose defense contract is violated while it compiles cleanly and runs correctly. Following our project constraint, we describe only the violated invariant (the falsification surface) and do not construct working exploits. The source-comment and RAG corpus is pinned to GCC 16.1 to keep evidence and reproduction anchored to one tree.

III. System Overview

DeFuzz shifts the paradigm from testing the program to testing the compiler itself. The orchestrator enumerates the entire checker catalog into a deterministic schedule; for each invariant a Generator agent produces a C seed; the seed runs through a hard-wired pipeline—routing → build → coverage → oracle—that yields an invariant verdict; a violation is handed to a minimizer agent, a non-violation goes through a feedback agent back to the next round. The full trajectory is versioned into a per-run checkpoint chain, so it is reproducible, ablatable, and auditable.

The system architecture is divided into two main components (Figure 4): a Go deterministic core that exposes both a gRPC adapter (deterministic build/coverage/oracle nodes) and an MCP adapter (read-only agent tools), and a Python orchestrator built on LangGraph that holds the control flow and the shared state. Both adapters read a single source of truth for checker metadata, so adjudication logic lives in exactly one place.

Figure 5 shows a single iteration end to end and forward-references §IV–§VII.

IV. Security Invariants for Compiler Defenses

A. What is a security invariant

A security invariant is a property a defense must satisfy for it to be effective; violating it means the mechanism has been silently weakened in the compiled artifact or at run time, whether or not the code itself contains an overt bug. Each invariant is recorded with a machine-verifiable statement and, separately, an observation field that names only the externally observable phenomenon of a violation—not how to detect it. The separation of the phenomenon from its detection recipe is deliberate: "the ___stack_chk_fail symbol is missing from the binary" is a phenomenon, whereas "look it up in the ELF .dynsym" is a detection mechanism that belongs to the oracle implementation.

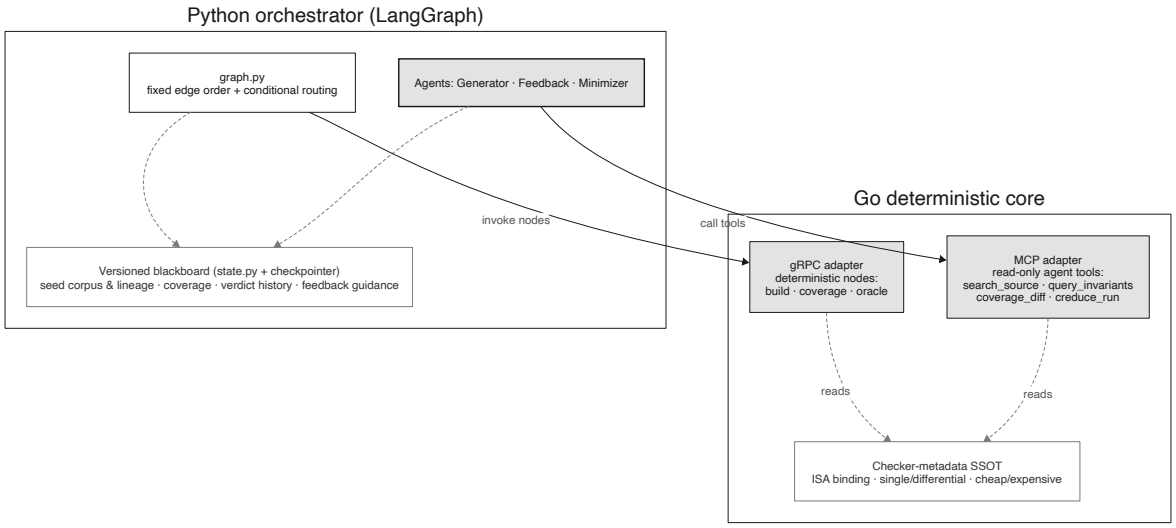

Fig. 4. DeFuzz two-part architecture: Python LangGraph orchestrator (control flow + blackboard) over a Go core with gRPC (deterministic nodes) and MCP (read-only agent tools) adapters.

B. Survey methodology and sources

The invariant archive draws on three source classes: (i) compiler source comments and assertions, (ii) compiler and ABI documentation, and (iii) confirmed historical bugs and their patches. Every record carries a uniform schema—ID, statement, compiler, version, target, source_kind, evidence, version_sensitivity, and observation—so that a downstream consumer (oracle, static analysis, CI) can decide how to use each invariant independently. Because the corpus is too large to read into a model context in one pass, we combine a segmented, per-mechanism manual survey (this section) with a retrieval-augmented generation pass that cheaply harvests the root-cause-homologous slice first (§V); the two are complementary rather than redundant.

C. A bottom-up taxonomy

Rather than imposing a top-down category scheme, we cluster the 468 catalogued invariants data-first to avoid preset bias, and read the root-cause families off the clusters (Figure 6). Three families recur across mechanisms and ISAs: (a) exit-time sensitive-register/state residue, in which a fast or exceptional exit bypasses the register/state clearing that a defense relies on; (b) stack-clash frame-size truncation, in which a forced integer narrowing flips a very large frame size negative and the probing loop is skipped; and (c) RISC-V large-address materialization, in which an unintended sign extension while splitting or synthesizing a large address/offset shifts the computed address.

D. Machine-checkable form: static vs dynamic

Each invariant is classified by whether its violation can be decided from the un-executed binary (symbols, sections, disassembly, strings)—static—or only by running it and observing exit status, output flags, or register/memory snapshots—dynamic. The choice is guided by four dimensions: observability, decisiveness, cost, and static/dynamic attribution. When an invariant has both a statically observable part and a more precise dynamic part, we split it into two checkers rather than forcing one classification.

V. Cross-Mechanism Invariant Generation

A. Motivation: why retrieval, and abstract-failure-mode transfer

The defense-relevant corpus—compiler source and its comments, ABI specifications, and confirmed historical bugs—is far too large to fit into a model context window, and an LLM asked to read it wholesale cannot reliably cluster the scattered, security-critical fragments that encode an invariant. Two complementary passes address this. The segmented, per-mechanism survey of §IV covers the corpus systematically but at high human cost. Retrieval-augmented generation supplies the cheap first pass: it uses a confirmed historical bug as a probe to pull in only the high-yield slice of the corpus that shares that bug's root cause, so that a defect homologous to a known one is unlikely to be missed even when the raw material never fits in context. The two passes are complementary—retrieval harvests the cost-effective, root-cause-homologous invariants first, and the segmented survey supplements the remainder.

Within the retrieval pass, we deliberately reject entity-driven retrieval (matching on API/symbol similarity): it would only rediscover the same mechanism. Instead we retrieve on the abstract failure mode—the shape of the root cause—so that a confirmed bug in one mechanism can point to an isomorphic site in another.

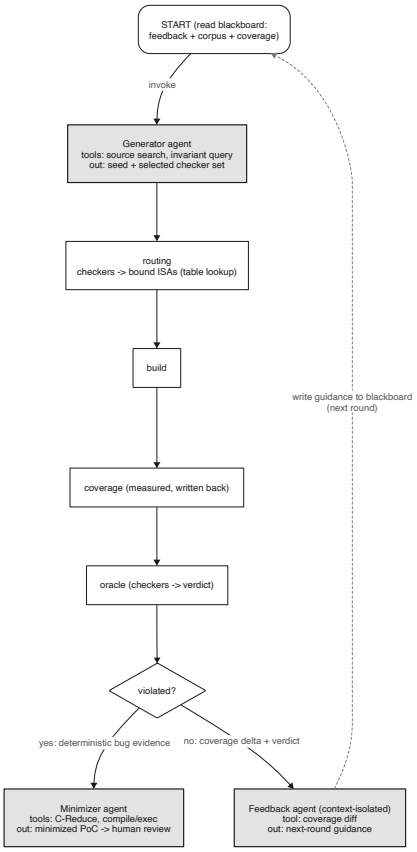

Fig. 5. Single-iteration data flow: START → generator → routing → build → coverage → oracle → route (violated → minimize; otherwise feedback → bump → next round).

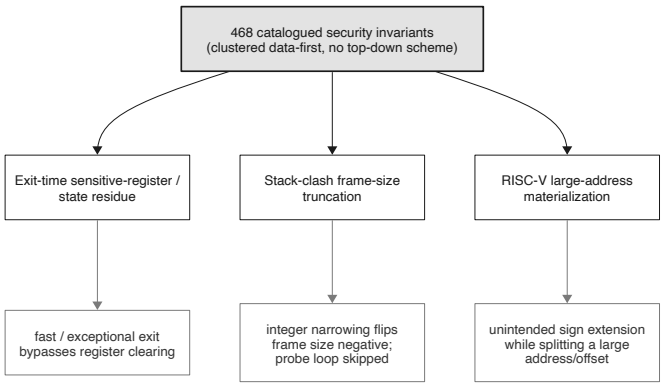

Fig. 6. Bottom-up root-cause families over 468 invariants; three families recur across mechanism and ISA.

B. Pipeline: distill → analogy → specialize → entailment

A confirmed root cause is first distilled into a mechanism-agnostic description; analogy is a semantic-isomorphism gate that decides whether a retrieved code block instantiates the same failure shape; specialize instantiates the invariant on the concrete target source; and entailment checks that the instantiated statement actually follows (Figure 7). The analogy gate is essential:

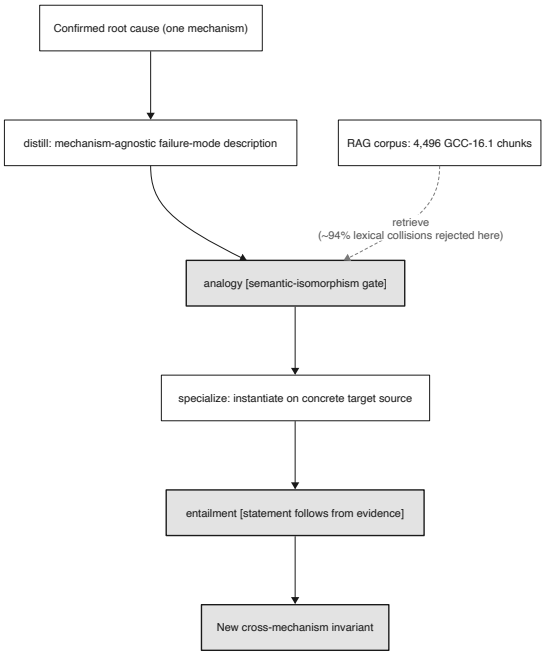

Fig. 7. SpecGen: distill (mechanism-agnostic root cause) → analogy (semantic gate) → specialize → entailment.

TABLE I
Cross-mechanism invariants by retrieval backend (24 probes, 4,496 GCC-16.1 chunks).

Category	Count	Note
Intersection (BM25 ∩ embed)	5	same seed & chunk, isomorphic state
BM25-only	4	embed missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	11	$9 + 7 - 5$; all is_novel

roughly 94% of BM25 hits are lexical collisions, and only the semantic gate keeps generation quality up.

C. Retrieval: BM25 and embedding are complementary

We run the same pipeline twice, changing only the retrieval backend: sparse BM25 and dense doubao-embedding-vision, over the same 24 probe seeds and the same 4,496 GCC-16.1 source chunks. BM25 excels on distinctive identifiers (ix86_zero_call_used_regs, SUBREG_PROMOTED, CONST_HIGH_PART); the dense retriever pulls in semantically isomorphic blocks whose lexical anchors are weak (the CMSE clear-set constructor, the RISC-V saturating truncation). Deduplicating by (seed_id, chunk_id) and statement, the two paths yield 5 in the intersection, 4 BM25-only, and 2 embedding-only, for 11 in the union (Figure 8); all 11 are marked is_novel after a novelty check (threshold 85.0). Table I summarizes.

D. Grounding gates and reproducibility

An invariant is accepted only after two static grounding gates confirm the retrieved evidence supports the

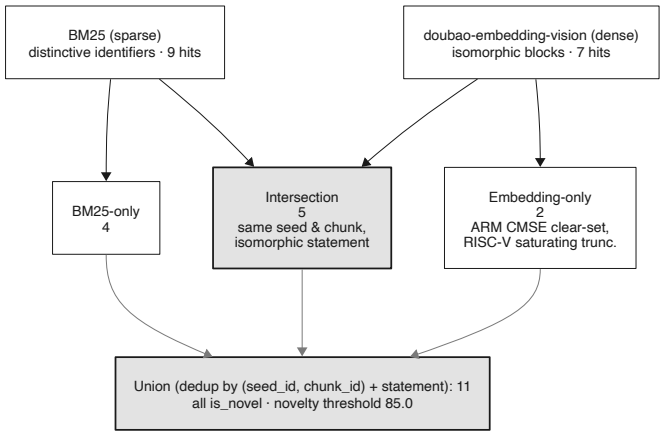

Fig. 8. Union/dedup of BM25 and embedding retrieval: 5 intersection, 4 BM25-only, 2 embedding-only, 11 union.

statement, and after the novelty check (threshold 85.0) confirms it does not duplicate an existing seed or manual rule. Because the dense retriever returns slightly different vectors per call, we cache query vectors keyed by query text so that top-k ordering—and hence the accepted set—reproduces across runs.

VI. Oracle: From Invariants to Verdicts

A. Checker design

Each invariant is realized by a checker that returns a four-state verdict (pass / fail / not-applicable / error); NotApplicable is returned honestly when the mechanism is off or the seed does not exercise it, and is ignored by the aggregation layer. Soundness is the central property: a bug is reported only when a deterministic checker fails or an execution differential reproduces, never on the strength of model output alone (Figure 9).

B. Mechanism aggregator and flag profiles

A per-mechanism aggregator combines checker verdicts: any Fail raises a mechanism-level violation carrying its evidence. On/off flag profiles let the oracle adjudicate differentially (defense enabled vs. disabled), and because the checkers key on binary and runtime signals rather than on the producing compiler, GCC and LLVM share the same checkers.

C. Checker metadata as a single source of truth

Each checker declares three metadata fields: which ISAs it binds, whether it is single-ISA or differential, and whether it is cheap or expensive. This metadata is the single source of truth read by both the gRPC and the MCP adapter, and it is what makes the ISA routing in §VII a table lookup rather than an agent decision.

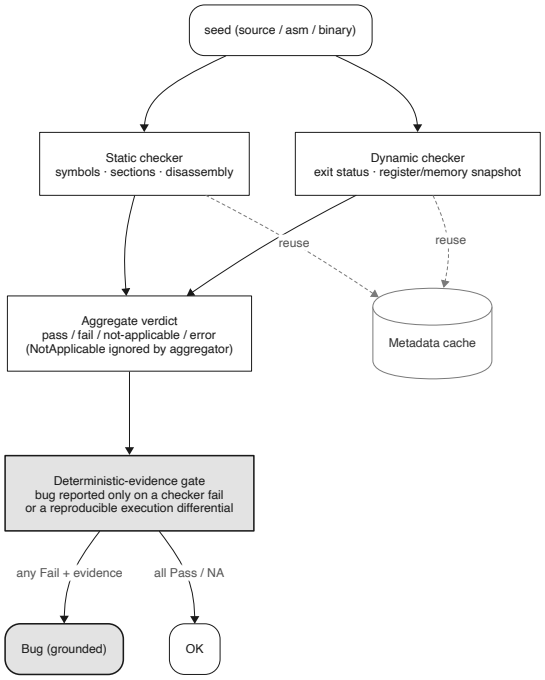

Fig. 9. Invariant → verdict flow: four-state checkers, NotApplicable transparency, and a deterministic-evidence gate before any bug is reported.

VII. Agentic Loop

A. Design principles

Four principles run through the design. (1) The pipeline is deterministic and hard-wired: generate → build → coverage → oracle → route, in a fixed order. (2) Agents are invoked only at fixed positions and do semantic judgment; the pipeline keeps control. (3) Agents never call each other directly—they communicate through shared state. (4) Correctness is held by deterministic evidence: the agent proposes, the oracle adjudicates.

B. Explicit orchestration and the blackboard

The linkage between agents is a real closed loop—feedback guides the next Generator round, the oracle adjudicates, a non-violation returns to feedback, a violation goes to the minimizer—but all of it flows through a versioned blackboard rather than direct agent-to-agent messages (Figure 10). The blackboard holds the seed corpus and lineage, cumulative coverage, the oracle verdict history, and the feedback agent's guidance. This design provides three critical properties that elevate agent-based bug hunting into a rigorous method: reproducibility (pin a blackboard version and any agent's input is fully determined), ablatability (each linkage edge can be switched off independently), and auditability (a bug claim traces back through the blackboard to a deterministic cause).

C. Three agents

The Generator (invoked at the start) carries read-only source-search and invariant-query tools and outputs

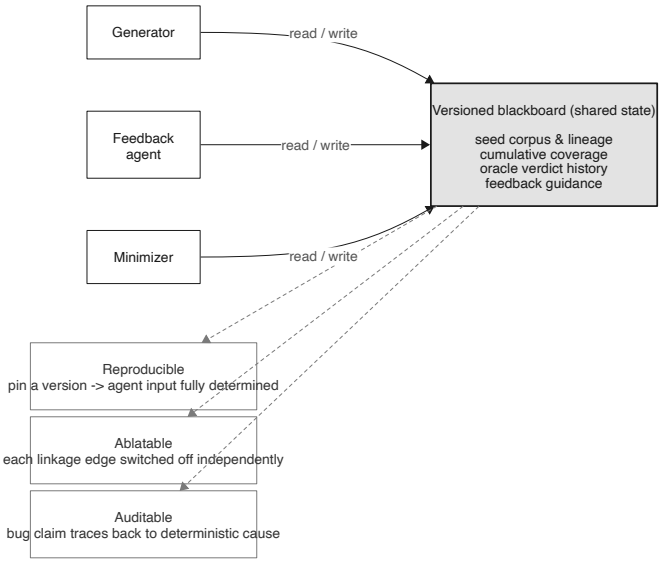

Fig. 10. The blackboard: all inter-agent linkage flows through versioned shared state, enabling reproduce / ablate / audit.

a new seed plus the checker set it selects; coverage is not an agent-reportable tool but is measured by the coverage node and fed back, so the agent cannot fabricate coverage. The feedback agent (invoked on a non-violation) is a context-isolated subagent that turns the coverage delta and the Pass/NotApplicable verdict into next-round guidance written back to the blackboard. The minimizer (invoked on a violation) shrinks the PoC with deterministic delta debugging (C-Reduce) as the workhorse and the LLM only for semantic guidance, so that reduction does not accidentally delete the structure that triggers the bug.

D. Checker-routed seeds, ISA-bound checkers

Instead of letting the agent pick an ISA, we statically bind each checker to its applicable ISA set, restricting the agent to the semantic task for which it is well-suited: "which checkers can this seed trigger?" Choosing a checker automatically claims all ISAs bound to it, so the agent never touches the raw ISA axis (Figure 11). Two backstops keep this an optimization rather than a correctness risk: cheap static checkers are always on and never enter the agent's decision, and the agent selects a superset ("keep everything not obviously impossible"), never a precise set. Differential checkers carry the "multiple ISAs + differential verdict" semantics, so once selected they force a full ISA fan-out and the cross-ISA silent-failure signal is preserved by metadata rather than by agent diligence.

VIII. Implementation

The Go core (cmd/defuzz-core) exposes grpc_server.go (build, coverage, oracle, and checker-metadata services) and mcp_server.go (the read-only tools search_source, query_invariants, coverage_diff, creduce_run, compile_exec); the oracle, checkers, and metadata SSOT

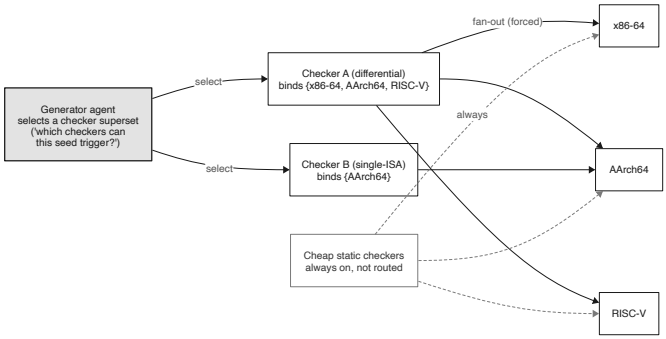

Fig. 11. Checker-bound ISA routing: the agent selects checkers; each checker fans out to its bound ISAs; cheap checkers always on.

live under internal/oracle. The Python orchestrator (defuzz_loop) holds graph.py (fixed edge order + conditional routing + enumeration cursor), state.py (the blackboard), audit.py (per-run directory + checkpointer), routing.py (the checker → ISA matrix expansion), the three agents, and the deterministic node wrappers. Each execution generates a dedicated per-run directory with checkpoints.sqlite (the versioned state chain) and manifest.json (git SHA, toolchains, checker catalog, LLM config, ablation flags), which the inspect / replay / tracebug subcommands consume for reproduction.

IX. Evaluation

Setup. Targets are an instrumented GCC 16.1, a cross-GCC, and LLVM; ISAs are x86-64, AArch64, RISC-V, and LoongArch, with cross-architecture execution under QEMU user mode. We answer five research questions.

a) RQ1 — Real bugs: Across GCC, LLVM, lld, and compiler-rt, DeFuzz produced 29 distinct silent-failure defects that cleared a five-item admission gate (concrete violated invariant, concrete impact, a source-level reason no later layer rescues the failure, at least one verbatim evidence site, and a reproduction recipe); one further candidate was retracted on closer inspection and is excluded from the count. Each defect is grounded in a deterministic binary or runtime signal rather than in model output. The corpus spans 17 defense mechanisms and 15 instruction-set targets, confirming that silent failures are not confined to a single cell of the mechanism × ISA matrix. Figure 12 breaks the corpus down by producing toolchain, defense mechanism, and affected ISA; GCC accounts for 19 defects and LLVM-family projects for the remaining 10, and ibt (Intel CET / RISC-V Zicfilp landing pads) is the most frequently violated mechanism. Seventeen defects are ISA-specific and twelve are generic. The defects concentrate on x86-64, RISC-V, and AArch64, the three targets with the densest control-flow-integrity surface. Table II summarizes the composition. Following the project's responsible-disclosure policy, findings remain private until the holder files upstream; at the time of writing one defect has been reported upstream and the

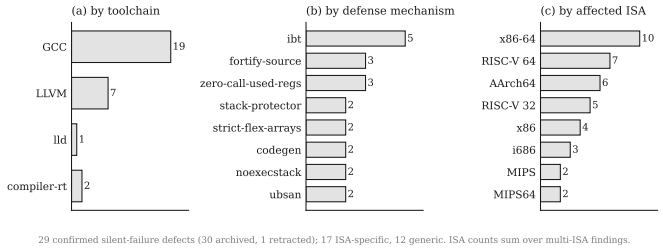

Fig. 12. Silent-failure defects found by DeFuzz across 29 confirmed findings: (a) by producing toolchain, (b) by defense mechanism, (c) by affected ISA. ISA counts sum over multi-ISA findings.

TABLE II
Composition of the DeFuzz silent-failure corpus (29 confirmed defects; one retracted candidate excluded).

Dimension	Count
Confirmed defects	29
produced by GCC	19
produced by LLVM / lld / compiler-rt	10
Distinct defense mechanisms	17
Distinct ISA targets	15
ISA-specific defects	17
Generic (ISA-independent) defects	12

rest are pending disclosure, so we report neither CVE identifiers nor upstream-confirmation counts here.

b) RQ2 — Invariant generation quality: The cross-mechanism pipeline produced 11 union invariants (5 intersection, 4 BM25-only, 2 embedding-only), all is_novel; the analogy gate filtered roughly 94% of BM25 hits as lexical collisions (Table I).

c) RQ3 — Ablation: We toggle coverage feedback, oracle grounding, and each inter-agent edge, and compare checker routing against a full Cartesian fan-out (Figure 13). [TBD: ablation numbers]

d) RQ4 — Explicit orchestration: Pinning a blackboard version reproduces a run's trajectory exactly; we compare stability and hit rate against free orchestration. [TBD: reproducibility numbers]

e) RQ5 — Agentic vs. fuzz loop: We measure the gain on "reaching silent-failure bugs" over the prior coverage-driven fuzz loop. [TBD: comparison numbers]

X. Discussion and Limitations

DeFuzz describes only violated invariants and does not build exploits; the RAG corpus is pinned to GCC 16.1; dense retrieval is non-deterministic and requires query-vector caching to reproduce; and checker routing is a performance optimization guarded by always-on static checkers, not a correctness mechanism.

XI. Related Work

Compiler bug finding. Csmith [1], YARPGen [2], and EMI [3] find miscompilations with crash or differential oracles, which by construction miss silent defense failures. Silent/security bugs in compilers. Studies such as "Silent Bugs Matter" [4] taxonomize compiler-introduced security bugs but do not build an agentic detector. LLM/RAG property generation. PropertyGPT [5] generates and verifies smart-contract properties via retrieval-augmented generation; SpecAuditor [6] detects specification violations by semantic generalization—we deliberately reject its entity-driven retrieval in favor of abstract-failure-mode transfer. LLM-agent fuzzing. AgentFuzz [7] applies directed grey-box fuzzing to LLM agents; in contrast, our agents are explicitly orchestrated rather than free-running.

[FIGURE PLACEHOLDER]
ablation

Fig. 13. Ablation: contribution of oracle grounding, coverage feedback, each inter-agent edge, and checker routing to hit rate/cost.

XII. Conclusion

An invariant-grounded oracle turns "agents can find bugs" from a one-off demo into a reproducible method for silent defense failures across the mechanism × ISA matrix. By pairing a sound oracle with an explicitly-orchestrated agentic loop and checker-bound ISA routing, DeFuzz makes such bugs both findable and auditable.

References

[1] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2011.
[2] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in Proceedings of the ACM on Programming Languages (OOPSLA), 2020.
[3] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2014.
[4] G. A. Di Luna et al., "Silent bugs matter: A study of compiler-introduced security bugs," in Proceedings of the USENIX Security Symposium, 2023.
[5] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.
[6] M. Lin and H. Chen, "SpecAuditor: Detecting specification violations via semantic generalization," in IEEE Symposium on Security and Privacy (S&P), 2026.
[7] F. Liu et al., "Testing the robustness of LLM-based agents via directed greybox fuzzing," in Proceedings of the USENIX Security Symposium, 2025.